

Trabalho 3 - CRUD por REST API

Matheus Berbel Barusso

Repositório completo do trabalho, com arquivos para replicação, disponível em: <https://github.com/MatheusBarusso/sistemas-distribuidos>

I. INICIALIZAÇÃO

Em um terminal utilizar o comando `uvicorn main:app --reload` para iniciar o WebServer Uvicorn, após isso acessar `http://127.0.0.1:8000/docs` para acessar página de controle da FastAPI. Frontend não foi implementado, portanto, as requisições são realizadas diretamente pela interface da FastAPI.

II. VALIDAÇÃO DE REQUISIÇÕES

Realizada por meio da biblioteca `pydantic`, mais especificamente com `BaseModel` e `Field`, que criam o modelo de dados e realizam a validação, respectivamente. Foi utilizada em `ProdutoEntrada`, `ProdutoAtualizacao` e `CategoriaEntrada` no arquivo `main.py`:

```
class ProdutoEntrada(BaseModel):
    codbar: int = Field(gt = 0)
    nome: str = Field(min_length = 1)
    estoque: int = Field(ge = 0)
    loc: int = Field(gt = 0)
    preco: float = Field(gt = 0)
    categoria: str = Field(min_length=1)

class ProdutoAtualizacao(BaseModel):
    nome: str = Field(min_length = 1)
    estoque: int = Field(ge = 0)
    loc: int = Field(gt = 0)
    preco: float = Field(gt = 0)
    categoria: str = Field(min_length=1)

class CategoriaEntrada(BaseModel):
    nome: str = Field(min_length=1)
```

- Código de Barras (codbar/int): Maior que 0
- Nome do Produto (nome/string): Ao menos 1 caractere
- Quantidade em Estoque (estoque/int): Maior ou igual a 0
- Localização / Corredor (loc/int): Maior que 0
- Preço (preco/float): Maior que 0
- Categoria (categoria/string): Ao menos 1 caractere
- Nome da Categoria (nome/string): Ao menos 1 caractere

III. ROTAS E VERBOS HTTP

Para as funções do CRUD foram designados os seguintes verbos e rotas HTTP:

Matheus Berbel Barusso, graduando em Engenharia de Computação da Universidade Tecnológica Federal do Paraná - UTFPR - Rua Cristo Rei 19, Vila Becker, Toledo-PR, CEP 85902-490 Brasil e-mail: (barusso@alunos.utfpr.edu.br).

Operação	Verbo HTTP	Rota
Create	POST	/produtos
Read	GET	/produtos/{codbar}
Update	PUT	/produtos/{codbar}
Delete	DELETE	/produtos/{codbar}

Para o gerenciamento de categorias foram designados os seguintes verbos e rotas HTTP:

Operação	Verbo HTTP	Rota
Create	POST	/categorias
Read	GET	/categorias/{nome}
Delete	DELETE	/categorias/{nome}
Listar por categoria	GET	/categorias/{nome}/produtos

IV. CÓDIGOS HTTP

Para os Erros e Acertos foram utilizados os seguintes códigos HTTP e suas utilizações:

- 200 OK: Busca, atualização, exclusão ou listagem feita com sucesso
- 201 Created: Produto ou categoria criada com sucesso
- 404 Not Found: Produto ou categoria não encontrada
- 409 Conflict: Produto/categoria duplicado ou Erro na exclusão de categorias que possuem produtos registrados.
- 422 Unprocessable Entity: Dados inválidos na requisição
- 500 Internal Server Error: Erro inesperado no servidor

V. NOVO RECURSO: CATEGORIAS

O novo recurso que complementa o CRUD implementado anteriormente são as Categorias. No banco de dados uma nova *table* "categorias" é criada em database caso não exista, com PK sendo uma id autoincrementada e possuindo além disso apenas o nome da categoria. Suas funcionalidades foram implementadas na mesma classe CRUD, com as funções:

```
def inserir_categoria(self, nome):
    return self.db.inserir_categoria(nome)

def buscar_categoria(self, nome):
    return self.db.buscar_categoria(nome)

def deletar_categoria(self, nome):
    return self.db.deletar_categoria(nome)

def listar_produtos_por_categoria(self, nome):
    return self.db....
    listar_produtos_por_categoria(nome)
```

Para funcionalidades como o *Create* do CRUD o banco de dados checa se a categoria inserida existe por meio de JOIN, portanto é necessário criar categorias antes dos produtos pois essa é uma informação necessária para a criação do produto

(retornando 404 caso contrário). As operações das Categorias são:

- Create: Cria categoria, funcionamento similar ao do CRUD, com diferença fundamental que PK não é escolhida na inserção e sim automaticamente no banco de dados.
- Read: Busca categorias pelo nome.
- Delete: Apaga categorias pelo nome.
- Listar por Categoria: Principal diferença em relação aos comandos do CRUD, essa funcionalidade encontra todos os produtos que possuem a Categoria inserida e retornam os valores de cada um deles.

VI. API UTILIZADA

Para o trabalho o framework FastAPI foi utilizado para implementar o webservice REST. As rotas estão definidas no arquivo `main.py` usando os decoradores do FastAPI para o CRUD:

- `@app.post("/produtos")`
- `@app.get("/produtos/{codbar}")`
- `@app.put("/produtos/{codbar}")`
- `@app.delete("/produtos/{codbar}")`

Para Categorias:

- `@app.post("/categorias")`
- `@app.get("/categorias/{nome}")`
- `@app.delete("/categorias/{nome}")`
- `@app.get("/categorias/{nome}/produtos")`